

Curso de Go

Aula 11 - Mutex, WaitGroups e Sync

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

código_[3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **O que são condições de corrida (race conditions)**
- **Como proteger dados compartilhados usando Mutex**
- **Como sincronizar goroutines com WaitGroup**
- **Como utilizar o pacote sync de forma segura**
- **Diferença entre concorrência segura e insegura**
- **Boas práticas no uso de sincronização**
- **Exemplos reais de concorrência em Go**
- **Parte Prática**



O problema da concorrência

- Quando várias goroutines acessam a mesma variável ao mesmo tempo, podem ocorrer problemas. Uma delas é a race condition
- Esse problema se dá quando duas goroutines modificam a variável ao mesmo tempo.
- As consequências são:
 - Dados corrompidos
 - Resultados imprevisíveis
 - Bugs difíceis de detectar



Race Condition - Exemplo Clássico

```
package main

import (
    "fmt"
    "time"
)

var contador int

func incrementar() {
    for i := 0; i < 1000; i++ {
        contador++
    }
}

func main() {
    go incrementar()
    go incrementar()

    time.Sleep(time.Second)

    fmt.Println(contador)
}
```



Mutex

- **Mutex significa: Mutual Exclusion**
- **Ele permite que apenas uma goroutine por vez acesse uma região crítica do código.**
- **O pacote utilizado é o “sync”**

Importação:

```
import "sync"
```

Estrutura Básica

```
var mu sync.Mutex
```

Métodos Principais

```
mu.Lock()
```

```
mu.Unlock()
```




Mutex

- **Funcionamento:**
 - **Lock trava o acesso**
 - **Unlock libera o acesso**
 - **Outras goroutines aguardam a liberação**
- **Mutex é utilizado para:**
 - **Variáveis compartilhadas**
 - **Estruturas globais**
 - **Escrita concorrente**
 - **Contadores**
 - **Cache em memória**



Protegendo dados com Mutex (exemplo)

```
package main

import (
    "fmt"
    "sync"
    "time"
)

var contador int
var mu sync.Mutex

func incrementar() {
    for i := 0; i < 1000; i++ {

        mu.Lock()

        contador++

        mu.Unlock()
    }
}

func main() {
    go incrementar()
    go incrementar()

    time.Sleep(time.Second)

    fmt.Println(contador)
}
```

- **Fluxo:**

- Uma goroutine chama Lock()
- As outras aguardam
- A variável é modificada
- Unlock() libera o acesso



defer com Unlock

- defer agenda a execução do `Unlock()` para o final da função, garantindo que o Mutex será liberado mesmo se ocorrer `return`, erro ou panic no meio do código.
- Ele evita bugs comuns de concorrência, como esquecer de chamar `Unlock()`, o que pode causar deadlocks e travar outras goroutines.
- O padrão mais usado em Go é:

```
mu.Lock()  
defer mu.Unlock()
```




defer com Unlock

- **Vantagens:**
 - **Código mais seguro**
 - **Evita esquecer Unlock**
 - **Melhor manutenção**
 - **Menor risco de deadlock**
- **Muito útil quando:**
 - **Existem retornos antecipados**
 - **Há tratamento de erros**
 - **O código é grande**



O que é WaitGroup

- **WaitGroup serve para esperar goroutines terminarem.**

Importação:

```
import "sync"
```

Estrutura Básica

```
var wg sync.WaitGroup
```

Métodos Principais

```
wg.Add()
```

```
wg.Done()
```

```
wg.Wait()
```

- Onde
 - Add adiciona tarefas
 - Done informa conclusão
 - Wait espera todas terminarem



Exemplo completo com WaitGroup

```
package main

import (
    "fmt"
    "sync"
)

func tarefa(id int, wg *sync.WaitGroup) {
    defer wg.Done()

    fmt.Println("Executando tarefa", id)
}

func main() {

    var wg sync.WaitGroup

    for i := 1; i <= 5; i++ {

        wg.Add(1)

        go tarefa(i, &wg)
    }

    wg.Wait()

    fmt.Println("Todas as tarefas terminaram")
}
```



Mutex + WaitGroup juntos

- O Mutex protege os dados compartilhados contra acesso simultâneo incorreto, enquanto o WaitGroup garante que todas as goroutines terminem antes do programa continuar.
- Juntos eles permitem concorrência segura e organizada, evitando tanto corrupção de memória quanto encerramento prematuro da aplicação.
- Essa combinação é muito usada em sistemas reais para controlar processamento paralelo com segurança, como servidores, APIs, filas de tarefas e processamento em lote.



Exemplo Mutex+Waitgroup

```
package main

import (
    "fmt"
    "sync"
)

var contador int
var mu sync.Mutex

func incrementar(wg *sync.WaitGroup) {

    defer wg.Done()

    for i := 0; i < 1000; i++ {

        mu.Lock()
        contador++
        mu.Unlock()
    }
}

func main() {

    var wg sync.WaitGroup

    for i := 0; i < 5; i++ {

        wg.Add(1)

        go incrementar(&wg)
    }

    wg.Wait()

    fmt.Println(contador)
}
```




RWMutex

- Muitas leituras simultâneas
- Apenas uma escrita exclusiva
- Ideal para:
 - Cache
 - Sistemas de leitura intensa
 - APIs
 - Configurações globais

Importação:
`import "sync"`

Estrutura Básica
`var mu sync.RWMutex`

Métodos Principais
`mu.RLock()`
`mu.RUnlock()`

`mu.Lock()`
`mu.Unlock()`



RWMutex - Exemplo de Código

```
var saldo int = 1000
var mu sync.RWMutex

func consultar() {
    mu.RLock()
    defer mu.RUnlock()

    fmt.Println(saldo)
}

func alterar() {
    mu.Lock()
    defer mu.Unlock()

    saldo += 100
}
```




sync.Once

- **sync.Once garante execução única.**
- **Muito usado para:**
 - **Singleton**
 - **Inicialização de banco**
 - **Configuração global**
 - **Carregamento de cache**

```
package main

import (
    "fmt"
    "sync"
)

var once sync.Once

func inicializar() {
    fmt.Println("Inicializando sistema")
}

func main() {

    for i := 0; i < 5; i++ {

        go once.Do(inicializar)
    }

    var entrada string
    fmt.Scanln(&entrada)
}
```



Problemas comuns com Mutex

- **Deadlock** : Acontece quando um Lock nunca é liberado.
- **Unlock duplicado** : Causa panic.
- **Esquecer Unlock** : Outras goroutines ficam presas.
- **Lock excessivo** : Muito lock reduz performance.
- **Boa prática:**
 - **Trancar apenas o necessário**
 - **Minimizar região crítica**
 - **Preferir operações rápidas**



Boas práticas de concorrência segura

- **Proteja dados compartilhados: Use Mutex ou RWMutex.**
- **Sempre sincronize goroutines: Use WaitGroup.**
- **Use defer para liberar recursos**
- **Evite regiões críticas grandes :Locks longos prejudicam performance.**
- **Prefira simplicidade: Concorrência complexa gera bugs difíceis.**
- **Teste race conditions**

The screenshot shows a Go IDE with a file named 'package main' in 'Untitled-1'. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then uses pointers to call these methods again. Comments in Portuguese explain the methods and the use of pointers. The IDE interface includes a sidebar with 'EXPLORER: TIMELINE' and a bottom panel with 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Mutex, WaitGroups e Sync